# Aetheris 项目进度报告与下一步规划

数据中台 · 个人主导 + AI 辅助构建 | 报告日期 2026-09-08 | 代码分支 main

## 一、项目概况

Aetheris 是一个 AI Native 数据中台：以 Java 21 + Spring Boot 3.4 微服务为后端、React 18 + Ant Design 5 为控制台，PostgreSQL 16 为元数据库，WSL Spark + Hive 为数仓底座。项目的核心命题不是「又一个数据平台」，而是验证一名工程师能否独立交付一套具备企业级工程纪律的系统。

技术规模：10 个 Spring Boot 微服务 + React 控制台；数仓五层 16 张表，约 130 万订单 / 320 万明细 / 180 个分区。差异化亮点：NL2SQL 三层防护（语句守卫 + 账号隔离 + 网络隔离）、语义层生命周期、企业级权限模型（RBAC + 资源授权 + 数据范围 + 权限版本）。

## 二、当前进度（截至 2026-09-08）

### 2.1 已交付的平台能力

| 模块 | 状态与说明 |
| --- | --- |
| 数据源接入 | 关系型 / Spark Thrift(Hive2 JDBC)，连接测试、只读查询、库表字段采集与注释读取 |
| SQL 开发 | 即席查询 + SQL 任务，NL2SQL 三层防护，语句守卫与只读语义 |
| 工作流调度 | 任务定义、DAG 依赖编排、业务日期参数、Quartz 调度、实例与日志 |
| 权限体系 | RBAC、资源授权、数据范围、权限版本，跨服务统一鉴权 |
| 数据治理 | 元数据、表级/字段级血缘、数据质量规则、告警、依赖影响分析 |
| 分析消费 | 语义模型、指标、数据集、看板、AI 智能问数（ChatBI / Agent） |

### 2.2 本轮工作（2026-09-07 ~ 09-08）

前端界面统一与优化

- 制定统一边距规范：在 tokens.css 引入页面节奏 token，.route-stage / PageHeader / Toolbar / DataSurface 全部改为 token 驱动，消除各页硬编码 margin 造成的上下错位。
- 风格统一：SQL 开发页、即席查询页从 Space 堆叠重构为 PageToolbar + DataSurface 标准框架，统一配色与交互组件（StatusBadge 等）。
- 首页（工作台）布局紧凑化：指标卡、服务状态、最近动态、侧栏在 100% 缩放下完整展示。
- 工作流 DAG 画布修复：修正节点 y 坐标重叠的布局 bug（按层叠加偏移），并收紧节点尺寸/间距，使画布在 100% 缩放内完整可见、无需整页滚动。

数据源连接健康检查（新能力）

- 后端新增 DataSourceHealthService、ConnectionHealth、ConnectionProbeFailure 等，独立于打开页面时的全库扫描，区分「连接探测」与「慢查询」。
- 新增数据库迁移 V17__datasource_connection_health.sql 与 auth 侧查询权限迁移 V4。
- 前端新增 useResponsivePageSize、ConnectionHealthStatus 等组件，实时展示探测状态。

Spark 链路现状审查

完成《Spark 现状与后续迭代计划》审查文档：确认「Spark SQL 数据源接入与 SQL 加工路径已有实现」，但「独立 Spark 作业管理尚未完成」，且当前 WSL Spark SQL

服务未运行、链路处于中断状态；给出 M0 – M6 里程碑与三个近期可验收交付。

### 2.3 代码基线状态

- 分支：main（另有 feat-unified-permission-model、feat-governance-permission-production 两个 worktree 分支）。
- 远端：GitHub（origin，https://github.com/mazhuce/Aetheris.git）与 Gitee（gitee，git@gitee.com:ybhkdsm/aetheris.git），两边已建立同步。
- 本轮新增未提交改动：前端界面统一、数据源连接健康检查、Spark smoke/恢复测试脚本、数据库迁移与相关文档。

## 三、下一步详细安排

总体顺序：恢复已有 Spark SQL → 修复安全与执行一致性 → 交付独立 Spark 批作业 → 补齐治理与可靠流水线 → 开放受管数据服务 → 增强语义与智能问数 → 按真实需求规模化与流式。

| 里程碑 | 目标 | 关键验收 |
| --- | --- | --- |
| M0 | 恢复并验收现有 Spark SQL 链路 | 平台身份 → 网关 → metadata → relay → Spark 真实查询成功；失联时提示具体依赖；重跑不重复写入 |
| M1 | 安全与统一执行基础（执行契约 + 原子抢占 + 身份/SQL 防护 + 提交未知恢复） | 并发提交仅一个执行者；响应丢失不产生第二个作业；审批后改动不可沿用旧批准 |
| M2 | 受管独立 Spark 批作业最小完整版本（PySpark/JAR 模板） | 真实 application ID、日志、取消、失联恢复、幂等重跑、未授权拦截 |
| M3 | Spark + 数仓加工 + 治理统一闭环 | 订单 → ODS → Spark 清洗 DWD → DWS/ADS → 质量门禁 → 指标审批 → 消费全链路 |
| M4 | 开放受管数据服务（应用注册、API 资产、配额、审计） | 未授权不可调用；撤权限时生效；轮换不泄露旧密钥 |
| M5 | 语义计算与智能问数增强（指标口径、Agent 权限继承） | 无连接放大；Agent 不返回无权限资产；固定问答评测集 |
| M6 | 按真实需求进入规模化与流式 | 先测瓶颈再投入；Kubernetes / CDC 独立验收 |

### 3.1 近期优先：三个独立可验收交付

- 交付 A：现有 Spark SQL 恢复 + 容器路径验收 + 状态真实性（先做 M0）。
- 交付 B：统一执行契约 + 原子抢占 + 身份/SQL 防护 + 提交未知状态恢复（M1 中阻塞 Spark 的部分）。
- 交付 C：受管独立 Spark 批任务 + application ID/日志/取消/恢复 + 完整界面（M2）。

工期应在 M0 环境恢复与 M2
集群技术验证后估算，当前没有足够证据承诺具体天数；每个交付以验收结果为进入下一步的条件。

### 3.2 每轮共同交付规则

- 先确认现有用户改动，按明确范围实现，不顺手重构无关模块。
- 每轮交付代码、迁移、API 契约、前端、负向测试、部署与恢复说明、运行验收记录。
- 构建成功 ≠ 运行验收通过；容器健康 ≠ 外部计算链路可用。
- 测试数据隔离；禁止清空现有数据库卷、重新格式化 HDFS 或覆盖仓库造数。
- 无权限、超时、失联、并发、撤权和重启恢复都是验收内容，不是以后再补的异常分支。
- 提交只包含本轮文件；GitHub 推送与实际部署按用户当轮授权执行。